

Dezibot - Steuerung mit Farben

Dokumentation im Rahmen des Oberseminars Roboter im Unterricht (C133)

Autor:innen: Anna Denzel und Alexander Reiprich

Betreuer: Prof. Dr. rer. nat. Jens Wagner

Projektidee

Im Rahmen des Oberseminars "Roboter im Unterricht" haben wir uns mit der Projektidee auseinandergesetzt, den Forschungs- und Lernroboter Dezibot in seiner Funktionalität so zu erweitern, dass er durch die Erkennung von Farben eine frei gestaltbare Strecke auf einem Bildschirm entlang laufen kann. Um dies zu erreichen, wurde ein mathematisches Modell aufgestellt, welches anhand von Lichtdaten die Position des Dezibots approximieren kann.

Projektaufbau

Für den Dezibot wurde eine Arena gebaut. Die Arena besteht aus einem Bildschirm, auf dessen Seiten ein LED-Band angebracht wurde. Auf dem Band sind 98 WS2812B LEDs angebracht. Die Arena ist 50 cm breit und 32 cm hoch mit je 29 LEDs auf den langen Kanten und je 20 LEDs auf den kurzen Kanten. Die LEDs werden mit Hilfe eines ESP32-Boards gesteuert, welches mit dem Dezibot kommuniziert. Der Bildschirm ist an einen Computer angeschlossen, über den eine Webapp läuft und den Bildschirm in blaue, rote, grüne und weiße Quadrate einfärbt. Über die Arena wurde ein imaginäres Koordinatensystem aufgespannt, mit der Position (0,0) in der unteren linken Ecke der Arena. Für eine Lokalisierung des Dezibots ist nicht nur seine Position im Koordinatensystem, sondern auch seine Ausrichtung sein Winkel relevant. In einem globalen Kontext wird dieser immer im Verhältnis zur positiven X-Achse gemessen. Das bedeutet, dass wenn der Dezibot horizontal nach rechts ausgerichtet ist, hat er den "globalen" Winkel 0° Grad, wenn er nach oben zeigt 270°.

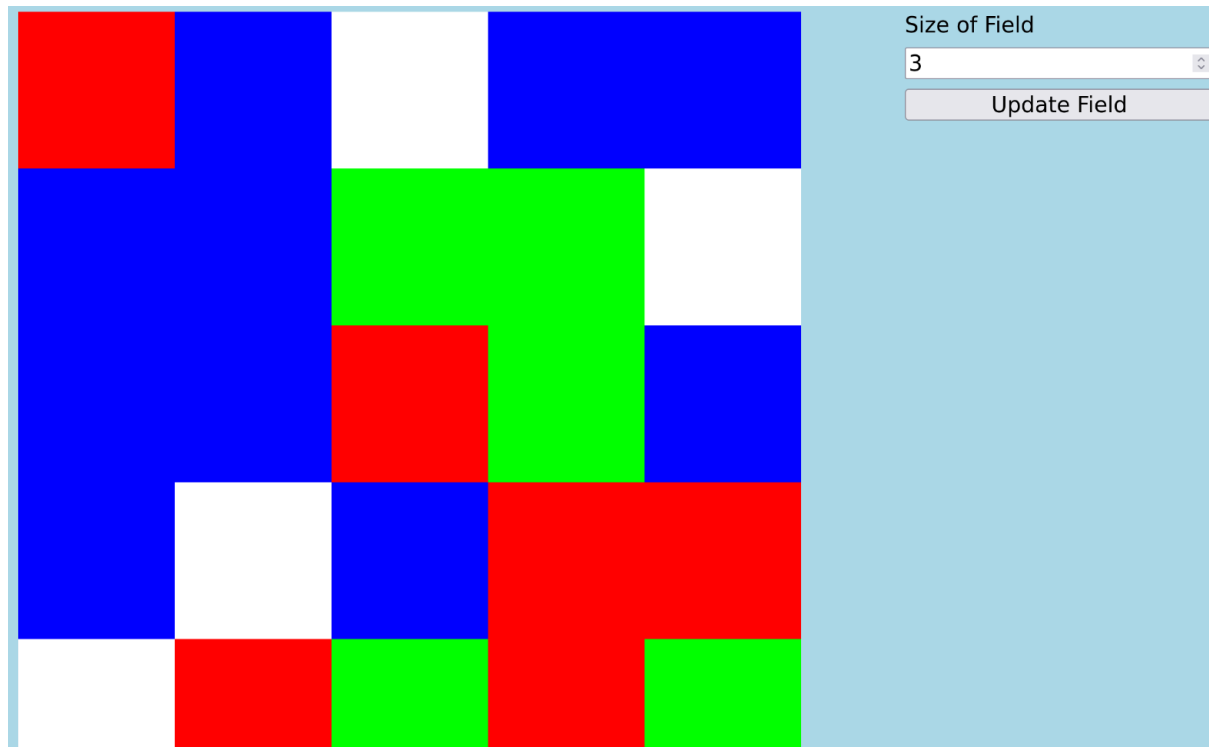
Ziel des Aufbaus ist, dass der Dezibot die Farbe des Quadrates auf dem Bildschirm unter sich erkennt und der Farbe entsprechend agiert. Um Drehungen akkurat bestimmen zu können, muss die Position und Ausrichtung bekannt sein. Da der Dezibot dies nicht verlässlich bestimmen kann, kommuniziert er mit einem ESP32-Board, welches LEDs am LED-Band an- und ausschalten kann. Dies kann der Dezibot nutzen, um basierend auf den Lichtdaten und einem mathematischen Modell seine Position und Ausrichtung zu bestimmen.

Color App

Der Dezibot wird durch die Farben gesteuert, die auf dem Bildschirm angezeigt werden. Dafür wurde eine einfache HTML-Seite gebaut, auf welcher ein Raster aus Quadraten

angezeigt wird. Die Anzahl der Quadrate kann der Nutzende durch ein Input-Element anpassen. Insgesamt kann ein Quadrat vier verschiedene Farben haben, welche die verschiedenen Aktionen repräsentieren, die der Dezibot ausführen kann.

Rot steht für eine Drehung entgegen des Uhrzeigersinns, Grün für eine Bewegung im Uhrzeigersinn und Blau für eine Bewegung geradeaus. Weiß dient als neutrales Element und stoppt jegliche Bewegung. Durch einen Klick auf ein Quadrat wechseln die Farben durch, sodass ein User eine Strecke aufbauen kann, welche der Dezibot folgen kann.



Figur 1: Screenshot der Webapp, die auf dem Bildschirm angezeigt wird

Kommunikation zwischen Dezibot und Board

Wie im Projektaufbau bereits beschrieben, ist das LED-Band an ein ESP32-Board angeschlossen, welches einzelne LEDs ansteuern kann. Für die Steuerung wurde die Arduino-Library "FastLED" verwendet. Da das Board nicht über die Bewegungsdaten des Dezibots verfügt, muss der Dezibot mit dem Board kommunizieren, um LEDs zu aktivieren.

Die Kommunikation wurde in der ersten Iteration implementiert und anschließend in einer zweiten Iteration erweitert und verbessert.

Zu Beginn des Projektes wurde die im Dezibot enthaltene Kommunikationsmethode `sendMessage` verwendet, welche auf der Library "painlessMesh" basiert. Hier wird für die Kommunikation ein Mesh aus den verschiedenen Teilnehmern erzeugt. Das Routing erfolgt über TCP/IP. Die Identifizierung der einzelnen Teilnehmer basiert dabei auf den Chip-IDs des zugrundeliegenden Systems. [<https://gitlab.com/painlessMesh/painlessMesh>] Um die

Kommunikation des Dezibots so einfach und einsteigerfreundlich wie möglich zu gestalten, wurde bei der `sendMessage`-Funktion die Library eingebunden, sodass Nutzende nur `sendMessage` und `onReceive` nutzen müssen, ohne direkt auf die Library selbst zuzugreifen. Während dies die Nutzung des Bots vereinfacht, ist es für unsere Zwecke nicht ausreichend, da wir mit der Dezibot-Methode auf einfache Strings begrenzt sind, ohne ein sinnvolles Kommunikationsprotokoll. Dazu kommt, dass "painlessMesh" von der Nutzung von Delays im Code abrät, da die Verbindung so instabiler wird und es zu Übertragungsverlusten kommen kann. Da wir jedoch auf Delays angewiesen sind, um ausreichend Lichtdaten sammeln zu können, wurde diese Art der Kommunikation im Verlauf des Projektes ersetzt.

In der zweiten Iteration wurde sich für eine direkte Einbindung des Moduls "ESP_NOW" entschieden. Hier basiert die Kommunikation auf WiFi, nutzt aber nur den MAC-Layer. Geräte können also Nachrichten an die MAC-Adresse eines anderen Geräts senden. Auch hier wird kein Router benötigt. Durch die Beschränkung einer Nachricht auf 250 Byte ist man hier im Umfang etwas beschränkt, da in diesem Anwendungsfall aber nur wenige, kurze Informationen übermittelt werden müssen, ist dies kein Ausschlusskriterium. Ebenfalls gab es weniger Probleme in der Nutzung von Delays, was ein finales Argument für die Verwendung von "ESP_NOW" war.

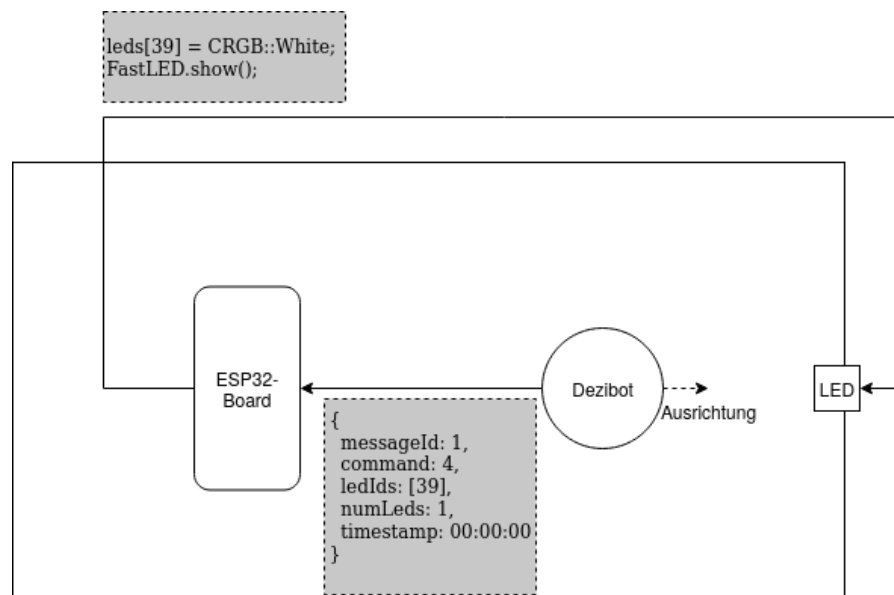
[<https://docs.espressif.com/projects/arduino-esp32/en/latest/api/espnow.html>]

Um eine sinnvolle Kommunikation zu gewährleisten, wurde ein Interface eingeführt, welches der übertragenen Nachricht Struktur gibt. In diesem Interface wurden die Attribute `messageId`, `command`, `ledIds`, `numLeds`, `timestamp` und `message` deklariert. `messageId` und `timestamp` tracken dabei jede verschickte bzw. erhaltene Message um Dopplungen zu vermeiden. `command` beschreibt die Aktion, die das Board ausführen soll, während `ledIds` und `numLeds` die genauen LEDs beschreibt, welche angesprochen werden sollen. `message` dient als optionales Debugging-Tool, bei dem der Dezibot Nachrichten an das Board schicken kann, sodass diese im Serial Monitor der Entwicklungsumgebung angezeigt werden können. Ein direktes Debugging des Dezibots über die serielle Schnittstelle ist nicht möglich, da dafür eine permanente USB-Verbindung benötigt wird, was die Lauffähigkeit des Dezibots negativ beeinflusst.

```
typedef struct {  
  
    uint32_t messageId;  
  
    uint8_t command;  
  
    uint8_t ledIds[MAX_LED_LIGHTS];  
  
    uint8_t numLeds;  
  
    uint32_t timestamp;  
  
    char msg[100]; // optional  
  
} RobotMessage;
```

Figur 2: Nachrichtenobjekt

Ein typischer Kommunikationsweg ist in der Figur 3 dargestellt. Hier wird auf dem Dezibot mit Hilfe des mathematischen Modells berechnet, welche LEDs er für eine genaue Ausrichtung benötigt. Diese Informationen werden mit der in Figur 2 dargestellten Struktur RobotMessage an das Board gesendet. Das Board verarbeitet den mitgelieferten Command und agiert basierend auf diesem Wert. Im Beispiel wird der Command 4 übertragen, was dem Einschalten einer LED entspricht. Nun wird über die FastLED-Library die mitgegebene LED als eingeschaltet markiert und das LED-Band mit den Änderungen aktualisiert. Auf dem Dezibot wird währenddessen ein kurzer Delay abgewartet, um Übertragungs- und Verarbeitungszeiten zu überbrücken, bevor eine Messung der Lichtwerte gemacht werden kann.



Figur 3: Ablauf der Kommunikation

Die Kommunikation verläuft unidirektional, da das Board selbst keine Berechnungen durchführt und nur als Ansprechpartner für die LEDs dient.

Mathematisches Modell

Ziel ist es, dass anhand der mit dem Tageslichtsensor gemessenen Lichtintensität (in Lux) die Position des Sensors herausgefunden werden kann. Hierfür ist ein mathematisches Modell notwendig, mit dessen Hilfe die Lichtintensität abhängig von der Position errechnet werden kann. Lichtintensität nimmt mit steigender Distanz zur Lichtquelle quadratisch ab (umgekehrtes Quadratgesetz)[StudySmarter, 2025].

$$E(lx) = I(cd)/d^2(m)$$

$E(lx)$ soll hierbei die Beleuchtungsstärke in Lux darstellen, die wir direkt mit der gemessenen Lichtintensität (ebenfalls in Lux) vergleichen können. $I(cd)$ ist die Lichtintensität in Candela und d ist die Entfernung des Sensors zur Lichtquelle in Metern. $I(cd)$ können wir abhängig

von einer normierten Lichtintensität `NORM_LIGHT` berechnen. Da der Sensor nicht immer genau auf die Lichtquelle ausgerichtet ist, müssen wir den Winkel des Sensors zur Lichtquelle ebenfalls mit bedenken. Dies können wir mit dem Lambert'schen Kosinusetz [Auersignal]. Damit kann $I(cd) = NORM_LIGHT * \cos(angle)$ aufgestellt werden. `NORM_LIGHT` ist eine konstante und normierte Lichtstärke und abhängig von der verwendeten Lichtquelle - in diesem Projekt die LEDs. Mit $\cos(angle)$ wird der Einfallswinkel des Lichts, also der Winkel von Sensor und LED relativ zur Blickrichtung der LED, mit in die Berechnung genommen. Das Koordinatensystem des Projekts ist außerdem in Zentimetern, statt Metern. Dies muss in der Formel ebenfalls berücksichtigt werden. Damit kann folgende Formel für die erwartete Lichtintensität in Abhängigkeit von der Distanz in Metern, dem Einfallswinkel des Lichts und dem Umgebungslicht aufgestellt werden:

$$I(d, angle, surLight) = surLight + NORM_LIGHT * \cos(angle) / d^2 + correction(d)$$

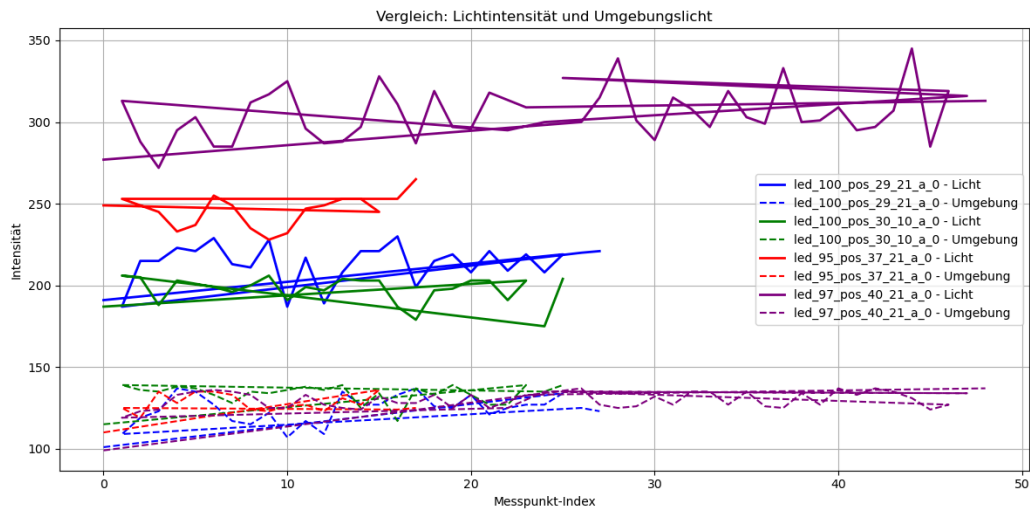
Hierbei mussten noch einige Konstanten auf das projektspezifische Setup angepasst werden. Die erwartete Lichtintensität entspricht in der Realität nur selten der tatsächlich Gemessenen, weswegen im Vergleich der beiden Werte einerseits eine geringe Abweichung in der Projektumsetzung als korrekt akzeptiert wird. Dieser akzeptierte Licht-Differenz ist in der Konstante `ACCEPTED_LIGHT_DIFF` festhalten. Andererseits wird die Differenz mit Hilfe der Korrekturfunktion `correction` möglichst weiter reduziert.

Normierter Lichtwert

Der normierte Lichtwert für dieses Projekt ließ sich anhand eines Testdatensatzes bestimmen. Hierfür wurde der Sensor des Dezibots genau 10 cm entfernt und direkt auf die LED ausgerichtet. Anschließend wurde das Umgebungslicht bestimmt und mehrere Lichtwerte gemessen. Zuvor aufgestellte Formel für die Lichtintensitätsberechnung wurde entsprechend umgestellt und für jeden Datenpunkt wurde die Formel, aufgelöst nach `NORM_LIGHT`, berechnet. Dabei wurde die Korrekturfunktion `correction` vorerst auf 0 gesetzt. Der Durchschnitt der Ergebnisse wird als Konstante `NORM_LIGHT` verwendet. Für dieses Projekt ist der Wert hierbei `1.958712`.

Akzeptierte Licht-Differenz

In der Umsetzung des Projektes ist eine Konstante `ACCEPTED_LIGHT_DIFF` definiert. Um diese Konstante möglichst passend für das Projekt Setup und die Anforderungen des Projektes auszuwählen, wurden mehrere Testdatensätze erhoben, für die mehrere Sätze an gemessener Lichtintensität und Umgebungslicht jeweils an der gleichen Position und für die gleiche LED betrachtet wurden. In folgender Grafik sind die erhobenen Testdaten visuell abgebildet.



Figur 4: Vergleich Lichtintensität und Umgebungslicht

Betrachtet man die Differenz unter den gemessenen Umgebungslichtern und Lichtern mit eingeschalteten LEDs anhand der 80-, 90- und 95 Perzentile, so kann man erkennen, dass die Differenz zwischen den Umgebungslichtwerten geringer ist, als die zwischen den Lichtwerten mit eingeschalteten LEDs.

Led id, Bot position	Umgebungslicht 80 Perzentil	Umgebungslicht 90 Perzentil	Umgebungslicht 95 Perzentil
Led_100, pos_29_21	18.0	24.0	26.0
Led_100, pos_30_10	12.0	18.0	21.0
Led_95, pos_37_21	12.0	15.0	17.0
Led_97, pos_40_21	10.0	12.0	16.0

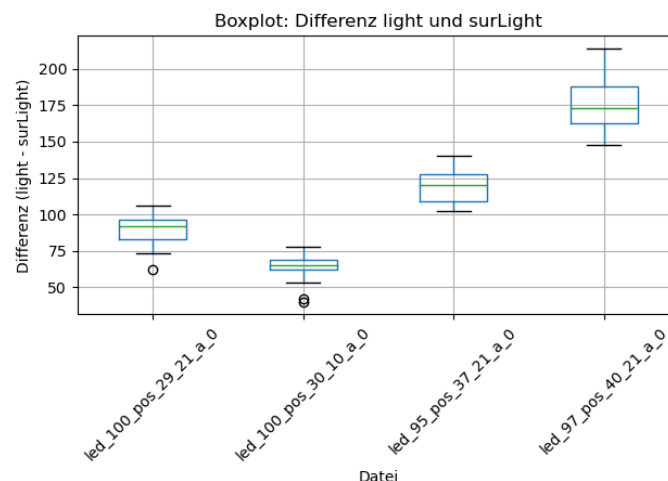
Led id, Bot position	Licht mit LED 80 Perzentil	Licht mit LED 90 Perzentil	Licht mit LED 95 Perzentil
----------------------	-------------------------------	-------------------------------	-------------------------------

Led_100, pos_29_21	24.0	32.0	34.0
Led_100, pos_30_10	16.0	21.0	25.0
Led_95, pos_37_21	18.2	21.0	25.0
Led_97, pos_40_21	29.0	37.0	44.0

Figur 5: Perzentile der Differenz von Umgebungslicht und Lichtwerten mit eingeschalteten LEDs

Hier lassen sich für die LED 100 mit Bot-Position (29,21) und LED 97 mit Bot-Position (40,21) größere Schwankungen erkennen. In Figur 4 lassen sich in beiden Fällen eine leichte Zunahme der Umgebungslichter feststellen. Daraus lässt sich schlussfolgern, dass die Messung des Umgebungslichtes möglichst zeitnah vor der des Lichtwertes mit eingeschalteter LED geschehen soll.

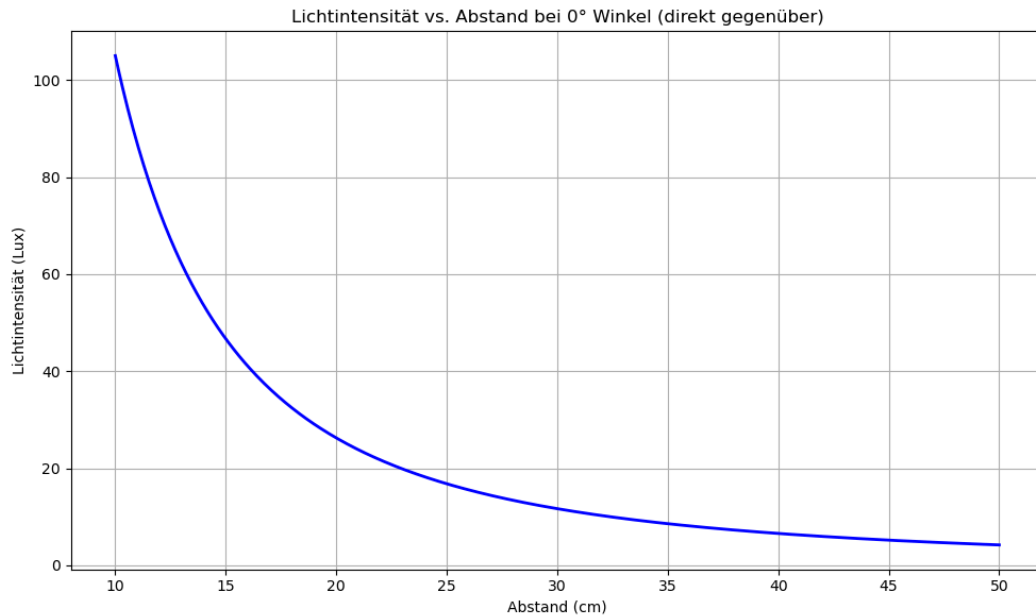
In Figur 5 ist erkennbar, dass ein höheres Umgebungslicht auch das direkt anschließend gemessene LED Licht tendenziell nach oben verschiebt. Deswegen wurde die Abweichung der Differenz von gemessenem LED Licht und Umgebungslicht betrachtet (Figur 5).



Figur 6: Boxplot mit den Abweichungen der Differenz zwischen Umgebungslicht und Licht mit eingeschalteter LED

Hier ist erkennbar, dass für eine größere Differenz auch eine größere Abweichung wahrscheinlicher ist – und dass ein Großteil der Daten sich in einem ± 35 lx Bereich bewegt (großzügig betrachtet).

Deshalb wurde für `ACCEPTED_LIGHT_DIFF` der Lichtwert 36 gewählt. Es wurde kein höherer Wert in Betracht gezogen, da sich bei größerer Distanz und/oder Winkel des Sensors zum Bot die Differenz zwischen Umgebungslicht und Lichtmessung mit angeschalteter LED ähnlich gering ist. Dies wird im folgenden Diagramm (Figur 7) veranschaulicht.

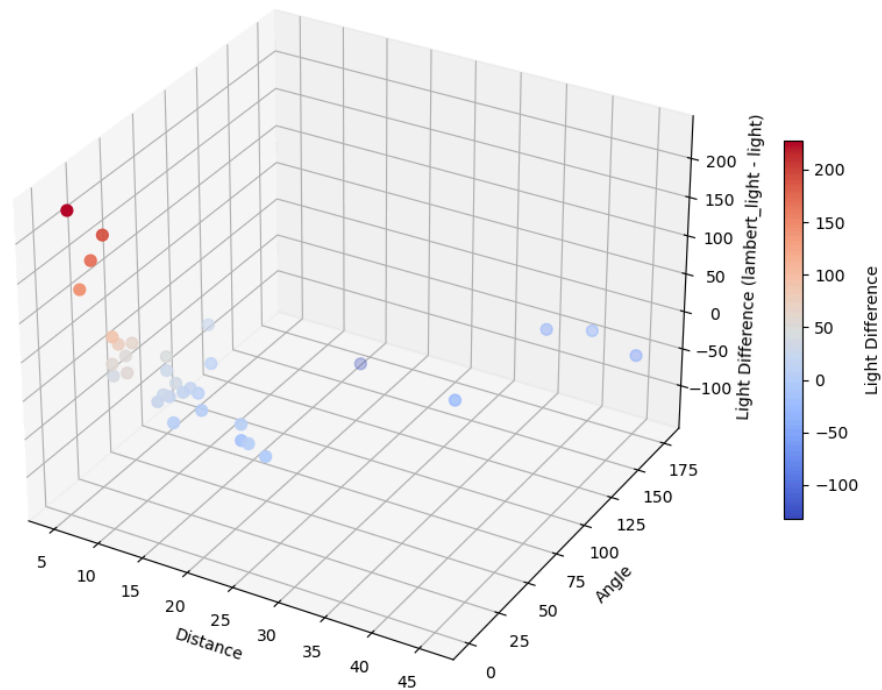


Figur 7: Bereinigte Lichtintensität abhängig von der Distanz bei Winkel 0°.

Korrekturfunktion

Für das Aufstellen der Korrekturfunktion wurden zuerst einige Testdaten manuell erstellt. Zuerst wurde versucht, durch lineare Regression eine Korrekturfunktion abhängig von der Distanz und dem Winkel zu machen. Dafür muss geprüft werden, wie sich die erwartete und gemessene Lichtintensität in Relation, abhängig von Distanz und Winkel, verhalten.

Abweichung vom Lambert-Modell

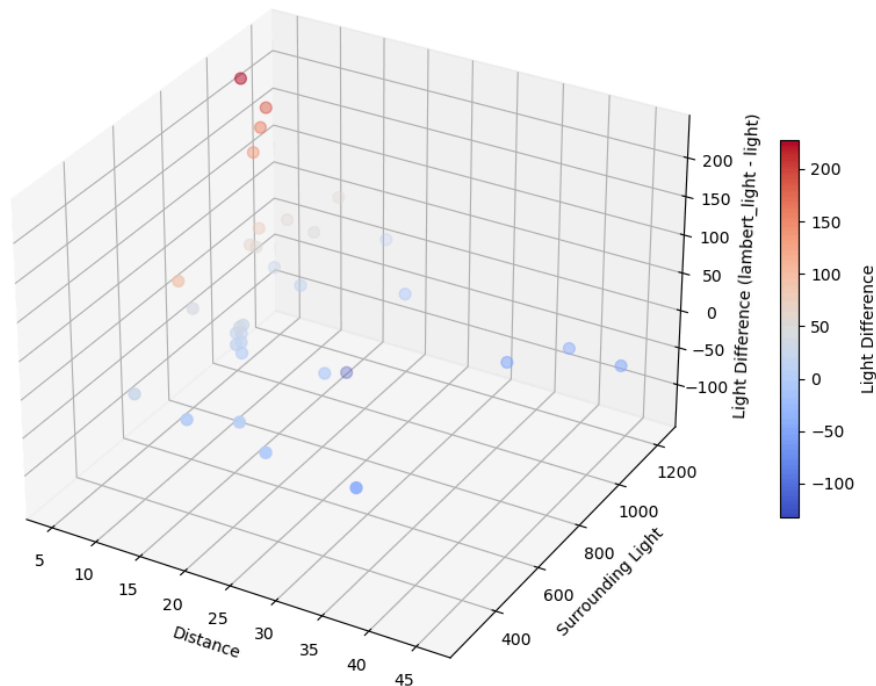


Figur 8: Differenz gemessenes und erwartetes Licht bzgl. Distanz und Winkel

Man kann hierbei zwei eindeutige Cluster erkennen - die tiefroten Punkte mit hoher Differenz und die hellroten bis blauen mit relativ niedriger Differenz. Aus den Testdaten wird sofort ersichtlich, dass die Punkte im tiefroten Cluster eine wesentlich geringere Distanz (≤ 7.3 cm) haben, während die blauen Punkte eine größere Distanz (≥ 10 cm) aufweisen. Anhand der Daten lässt sich auch erkennen, dass die Differenz bei steigender Distanz tendenziell geringer wird. Der Winkel scheint (für das menschliche Auge) erstmal keinen signifikanten Einfluss auf die Differenz zu haben.

Bei der Wahl der akzeptierten Licht-Differenz war schon erkenntlich, dass ein höherer gemessener Lichtwert mit größeren Lichtwertschwankungen einhergeht (bei gleicher Position und Ausrichtung von Sensor und Lichtquelle). Die Testdaten für die Korrekturfunktion bestätigen dies. Gleichzeitig ist mit Figur 9 aber auch erkenntlich, dass die Differenz auch bei einem hohen Umgebungslichtwert gering sein kann und dies für eine größere Entfernung auch der Fall ist. Deswegen wurde das Umgebungslicht in der Korrekturfunktion nicht weiter betrachtet.

Abweichung vom Lambert-Modell

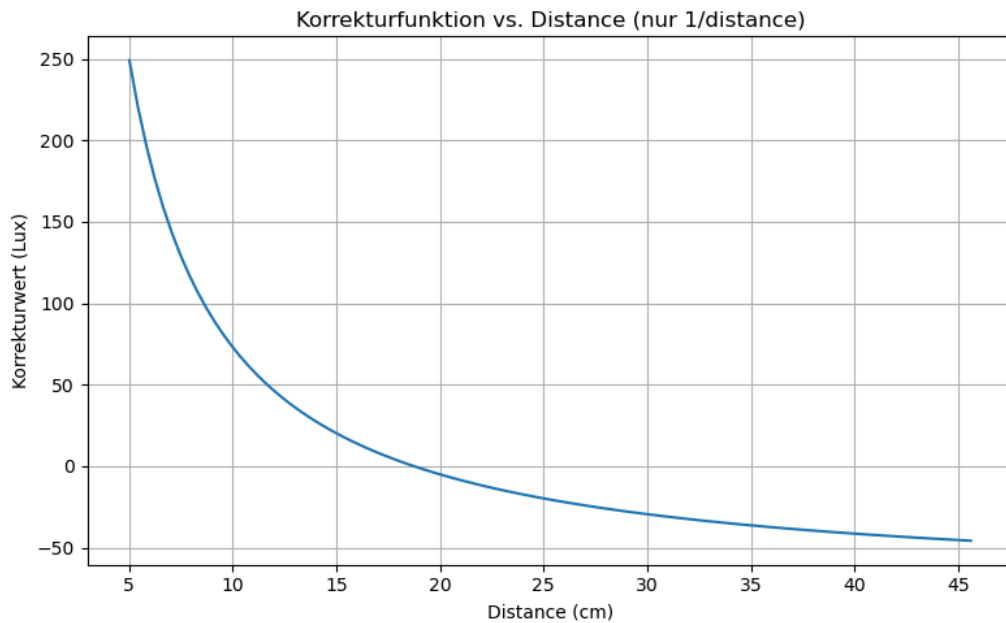


Figur 9: Differenz gemessenes und erwartetes Licht bzgl. Distanz und Umgebungslicht

Im nächsten Schritt wird eine polynomiale Regression durchgeführt. Dabei wurden Regressionsfunktionen verschiedener Grade und mit unterschiedlichen Gewichtungen der 2 relevanten Variablen - Winkel und Abstand - in Betracht gezogen und die Konstanten der Regressionsfunktion möglichst klein gehalten. Im Vergleich, hat eine Korrekturfunktion zweiten Grades, die nur das Inverse des Abstands in die Kalkulation mit einbezieht, am Besten funktioniert. Konkret ist die implementierte Korrekturfunktion:

```
float correction(float distance) {  
    double inv_distance = 1.0 / distance;  
    double x0 = (inv_distance - 0.0759972064) / 0.0381503382;  
    double result = 0.8643417010 * x0 + 0.0301542091 * x0 * x0;  
    return - (result * 69.1178872422 + 34.6799148574;  
}
```

In Figur 10 wird das Korrekturpolynom visuell als Graph dargestellt. Wie man in Figur 9 erkennen kann, muss das rote Cluster nach unten und das blaue nach oben korrigiert werden, weswegen das Ergebnis in der correction Funktion negiert zurückgegeben wird.



Figur 10: Korrekturpolynom 2. Grades abhängig vom Inversen der Distanz

Referenzprojekte

Im Rahmen der Entwicklung wurde sich an verschiedenen anderen Projekten orientiert, bei welchen ähnliche Probleme bewältigt werden mussten. Hierbei soll der Fokus auf zwei spezifischen Bereichen liegen - der Approximierung der Position des Roboters im Raum und der Bewegungskorrektur des Roboters.

Das bekannteste Projekt ist das ähnlich benannte Forschungsprojekt "Kilobot" der Harvard Universität. Der Kilobot ist ein kleiner, kostengünstiger Roboter, der für den Einsatz in Schwarmexperimenten entworfen wurde. Bis zu 1000 Roboter können gleichzeitig als Schwarm agieren und so das Verhalten von Ameisen, Bienen o.Ä. nachstellen. Die Hardware des Kilobots ist dabei ähnlich, wenn auch etwas simpler als die des Dezibots - der Kilobot ist verwendet einen 8Mhz Atmega328 Mikroprozessor mit 32 Kilobyte an Memory, was verglichen mit dem ESP32 des Dezibots nur sehr einfache Programme zulässt. Ebenfalls ist der Kilobot beschränkt auf zwei Vibrationsmotoren, drei dünne Beine, eine Status-LED und einen Infrarot-Sender und -empfänger. Ein WLAN-Modul oder ein Tageslichtsensor wie beim Dezibot fehlen.

Die Kommunikation des Kilobots erfolgt über das Infrarotmodul. Dabei sendet ein Kilobot ein Infrarotsignal auf den Boden in die Richtung eines anderen Kilobots. Dieser kann dann mit einem Empfänger das Signal auslesen. Diese Art der Kommunikation ist begrenzt auf 10 Zentimeter und ist anfällig für andere Signale oder Rauschen von außerhalb, weshalb diese Verständigung zwar für das Konzept des Schwarmgedanken ausreichend, aber für dieses Projekt nicht verwendbar ist.

Drei dünne Beine sowie zwei Rotationsmotoren ermöglichen dem Kilobot eine Form der Fortbewegung. Hier gilt das selbe physikalische Konzept wie bei dem Dezibot, was auch

beim Kilobot zu einem instabilen, nicht-deterministischen Laufweg über längere Strecken führt. Gelöst wurde dieses Problem, indem sich immer an nahegelegenen Kilobots orientiert wurde. Dieses Vorgehen ermöglicht dem Kilobot einen stabilen Laufweg, vorausgesetzt, dass sich in der Nähe des Weges weitere Kilobots befinden. Für unseren Anwendungsfall ist dieses Konzept ebenfalls nicht geeignet, da die räumliche Begrenzung von 10 Zentimetern zu gering ist, um die Länge und Breite des Bildschirms abzudecken [Rubenstein et al., 2014].

Ein Dezibot-Projekt, welches Infrarot zur Lokalisierung und Approximation im Raum nutzt, ist das Projekt "embedded-chess-pieces". Hier verkörpert der Dezibot eine Schachfigur auf einem Schachbrett. Ziel des Projektes ist es, die Bewegung der Figur mit dem Dezibot auf dem Brett nachzustellen. Der Dezibot muss also gewisse Distanzen und Drehungen in einem ihm unbekannten Raum ausführen, vergleichbar mit diesem Projekt.

Um die Position des Dezibots im Raum zu erhalten, wurde an einer Ecke des Schachbretts ein Dezibot in einem 45° -Winkel in Richtung des Brettmittelpunkts platziert. Dieser wird als Beacon bezeichnet und dient als Orientierungspunkt für den Dezibot auf dem Brett. Der Beacon sendet Infrarotsignale aus, die der andere Dezibot messen kann. Anhand der vier Sensoren an den Seiten des Dezibots ist es möglich, den Winkel zwischen Dezibot und Beacon zu berechnen, um so präzise Rotationen durchführen zu können.

Da bei diesem Projekt eine genaue Positionierung auf dem gesamten Schachbrett nicht nötig war, wurde sich auf eine relative Berechnung der Position durch Prüfen der Übergänge von schwarzen zu weißen Feldern beschränkt.

Bei längeren Bewegungen und dem Drift des Dezibots wurde sich dafür entschieden, kurze, kleine Bewegungen in die gewünschte Richtung zu machen, was das Risiko einer Winkelveränderung verringert, aber nicht eliminiert.

Die Idee eines Beacons wurde für dieses Projekt ebenfalls diskutiert, wurde schlussendlich jedoch verworfen, da eine präzisere Positionsapproximation benötigt wird, welche mit diesem Verfahren nicht erreichbar ist [Rohrbach & Schramm, 2025].

Ebenfalls betrachtet wurde ein Ansatz von Chew et al. bei welchem ein Roboter ausschließlich durch eine Photodiode gesteuert und lokalisiert wurde. Das VLP (Visible Light Positioning) basiert in diesem Paper auf einer Konstruktion mit vier LEDs über einer Fläche, auf der sich ein Roboter befindet. Auf diesem Roboter befindet sich eine Photodiode, welche die Lichtstärke zu jeder LED misst. Die Position der LEDs ist dabei bekannt. Nun wird diese Lichtstärke mit einer inversen Lambertschen Funktion in die jeweilige Entfernung umgewandelt, um anschließend mit Hilfe von Trilateration die Position des Roboters im Raum bestimmen zu können.

Chew et al. beweisen damit, dass es möglich ist, einen Roboter nur durch VLP zu lokalisieren und zu steuern. Übertragbar ist dieses Verfahren jedoch nicht auf unser Projekt, da der Dezibot nur eine Photodiode mit Ausrichtung nach vorne besitzt. So kann kein Licht von der Rückseite des Dezibots empfangen werden, was eine Positionierung mit diesem Konzept stark einschränkt [Chew et al., 2024].

Probleme

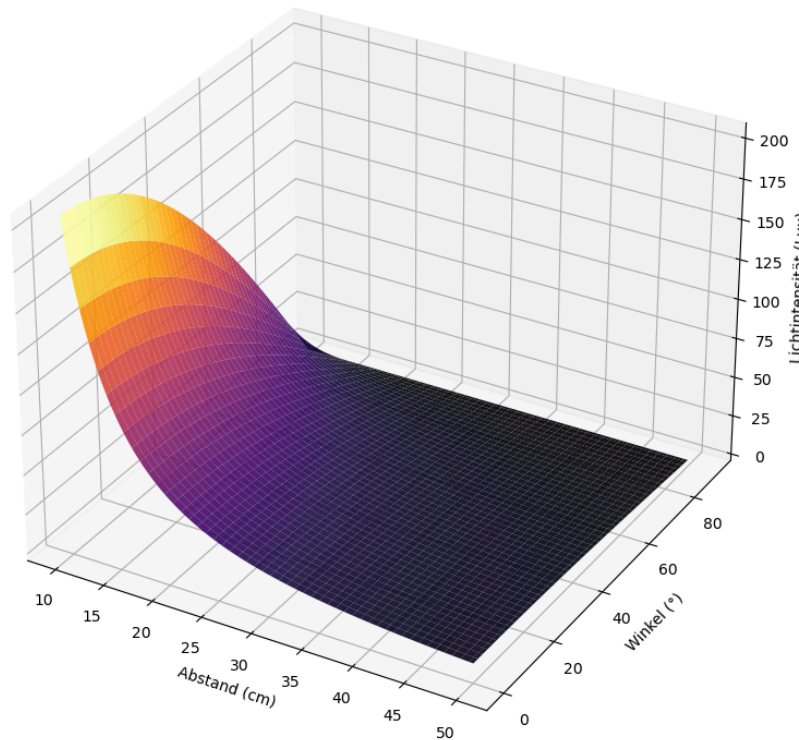
Die Probleme dieses Projektes lassen sich in drei Bereiche einteilen. Als erstes die genaue Lokalisierung des Dezibots auf dem Bildschirm. Angrenzend dazu stehen die eingeschränkten Bewegungskapazitäten des Dezibots, welche eine präzise Bewegung nicht verlässlich geschehen lassen. Abschließend steht die Berechnung der Korrekturfunktion, welche Teil beider Probleme ist.

Während es mehrere Ansätze für die Approximation der Position des Dezibots gab, konnte keine verlässlich gute Ergebnisse erzielen. Die Ortung des Dezibots durch die LEDs funktioniert in der Theorie, scheitert jedoch in der Praxis an dem verbauten Lichtsensor, welcher ab einer Entfernung von etwa 20 Zentimetern keinen Unterschied mehr zwischen Umgebungslicht und eingeschalteter LED erkennt, was eine präzise Ortung nicht möglich macht.

Ebenfalls bedeutet dies, dass eine Korrektur der Vorwärtsbewegung ab einer gewissen Distanz zu dem LED-Streifen nicht gewährleistet werden kann, da der Dezibot sich nicht an dem Licht einer LED orientieren kann. Durch die nichtdeterministische Charakteristik der Fortbewegungsmethode ist es also in einem Großteil des Bewegungsbereichs des Dezibots weder möglich, die Positions Berechnung akkurat zu halten, noch darauf zu vertrauen, dass der Dezibot den vorgesehenen Weg läuft.

Neben diesen hardwarebezogenen Problemen traten ebenfalls Probleme im mathematischen Modell auf. Hier geht es um die Korrekturfunktion, welche die gemessenen Lichtwerte so korrigiert, dass sie näher an dem berechneten Erwartungswert sind. Sollten die korrigierten Werte dann nah genug an den mathematischen Ergebnissen liegen, kann davon ausgegangen werden, dass sich der Dezibot an dieser Position mit dieser Ausrichtung befindet. Diese Korrekturfunktion basiert auf Testdaten, welche händisch angelegt wurden. Dadurch, dass sich die Testdaten in den Werten jedoch stark unterscheiden, ist es nicht möglich gewesen, eine verlässliche Korrekturfunktion zu definieren, welche jede Position mit jedem Winkel zu den berechneten Lichtwerten korrigiert. Dies führt dazu, dass sowohl die genaue Berechnung der Position als auch eine verbesserte Bewegung nicht möglich ist.

Zusätzlich traten Probleme bei der Definition des Akzeptanzbereichs der Lichtdifferenz zwischen erwarteter und tatsächlicher Position auf. Die natürlich auftretenden Lichtschwankungen bei stillstehendem Dezibot sind vergleichbar mit den Lichtwerten von benachbarten Positionen. Dies führt dazu, dass man nicht klar unterscheiden kann, an welcher Stelle der Dezibot steht. Dieses Problem tritt jedoch erst bei einer bestimmten Entfernung von etwa 20 Zentimetern von der leuchtenden LED auf. In Figur 12 ist dieses Problem und Verlauf von Lichtintensität abhängig vom Abstand grafisch dargestellt.



Figur 12: messbare direkte Lichtintensität abhängig von Distanz und Winkel

Ausblick und Fazit

Während der Bearbeitungszeit dieses Projekt wurden viele Ansätze ausprobiert, welche sowohl Probleme gelöst, aber auch neue erschaffen haben. Dass die Aufgabe schlussendlich als nur teilweise erfolgreich betrachtet werden kann, gibt Raum für neue Projekte, die auf den Erkenntnissen von diesem aufbauen.

Beispielsweise wäre eine Kombination der LEDs und des Beacon-Konzepts denkbar, um eine präzisere Ortung des Dezibots zu ermöglichen, oder die Gegebenheiten (Bildschirm, LED-Band, Lichtverhältnisse, ...) grundlegend zu ändern. Die oben beschriebenen Referenzprojekte können hier als Inspiration oder Denkanstoß dienen, um Neues auszuprobieren und das Projekt voranzutreiben.

Quellen

- [Rubenstein, Michael, et al., 2014] Rubenstein, Michael, et al. "Kilobot: A low cost robot with scalable operations designed for collective behaviors." *Robotics and Autonomous Systems* 62.7 (2014): 966-975.

- [Auersignal]
<https://www.auersignal.com/de/technische-informationen/optische-signalgerate/lichtstarke/>, aufgerufen am 22.07.2025
- [StudySmarter, 2025]
<https://www.studysmarter.de/ausbildung/ausbildung-in-der-medizin/feinoptiker-ausbildung/lichtintensitaet/>, zuletzt modifiziert am 13.04.2025, aufgerufen am 22.07.2025
- [Rohrbach & Schramm, 2025] Rohrbach, Ines; Schramm, Nico. "Embedded Chess Pieces - Projekt-Dokumentation für das Modul C999 Softwareentwicklung für Eingebettete Systeme" <https://github.com/embedded-chess/doc/releases>, zuletzt modifiziert am 26.03.2025, aufgerufen am 22.07.2025
- Chew, M.-T., Alam, F., Noble, F. K., Legg, M., & Gupta, G. S. (2024). Visible Light Positioning-Based Robot Localization and Navigation. *Electronics*, 13(2), 368.
<https://doi.org/10.3390/electronics13020368>